

Arquiteturas

Multi-Agent (PT-5)

Arquitetura de Sistemas com IA · Módulo 3.5

Prof. Ahirton Lopes, Ph.D.

A Arquitetura Completa

Agente	Padrão	Falha (CAP)	Compensação (Saga)
Protocolo	Sequential (1º)	Retry 3x, timeout 30s	Versiona (v1→v2)
ICF	Parallel + Handoff	Retry 2x, timeout 45s	Regenera seção afetada
CSR	Parallel	Retry 2x, timeout 60s	Aciona compensação a montante
Supervisor	Supervisor	N/A — coordena, não falha sozinho	Decide qual compensação disparar

Cada linha desta tabela resume uma decisão que você já justificou em algum vídeo anterior deste módulo — nada aqui é novo, é integração.

TrialForge: Fila de Mensagens em JS

Comunicação assíncrona entre os três agentes

```
const filaDeMensagens = new EventEmitter();

// Agente Protocolo publica quando termina (assíncrono)
async function agenteProtocolo(estudo) {
  const criterios = await gerarCriterios(estudo);
  filaDeMensagens.emit('protocolo:pronto', { versao: 1, criterios });
}

// ICF e CSR escutam o mesmo evento — Parallel após Sequential
filaDeMensagens.on('protocolo:pronto', async (msg) => {
  const [icf, csr] = await Promise.all([agenteICF(msg.criterios),
    agenteCSR(msg.criterios)]); // agora sim, em paralelo
  filaDeMensagens.emit('supervisor:verificar', { icf, csr });
});
```

Cada agente publica um evento quando termina — ninguém espera de forma bloqueante.

Missão Prática #03

01

Mapeie os padrões do seu protótipo

Do protótipo que você desenhou no Módulo 2, identifique qual dos 6 padrões de orquestração se aplica a cada dependência entre os agentes.

02

Declare falha e compensação

Para cada agente, defina timeout/retry (CAP) e, para cada etapa sequencial, a ação compensatória (Saga) caso uma etapa posterior falhe.

03

Implemente a comunicação assíncrona

Em JavaScript, usando fila de mensagens (EventEmitter ou similar), implemente a comunicação entre pelo menos dois dos seus agentes.

Entrega: diagrama dos padrões + tabela de falha/compensação + código JS da comunicação + reflexão final, num único repositório ou arquivo.

Os Agentes Coordenam. E os Padrões Internos?

O Módulo 3 resolveu COMO agentes se comunicam entre si. Mas cada agente individual ainda usa RAG genérico, chama sempre o mesmo modelo caro, e nunca reaproveita uma resposta já dada antes.

Módulo 4 → Padrões de Design AI-Específicos: RAG avançado, roteamento inteligente de modelo, cache semântico, e Human-in-the-Loop com thresholds de confiança.